September 4, 2026 at 23:51

1. Introduction. Exercise 7.2.2.1–334 is about false fronts. The seven Soma pieces cannot really build the W-wall of Fig. 75—exercise 326 proves that by a neat factoring argument—but they can build something that *looks* like the W-wall when you stand in front of it, the way a Hollywood set looks like a town. Knuth asks for all such façades of three shapes: the W-wall, the X-wall, and the plain $3 \times 3 \times 3$ cube.

His answer says the count is 282 for the W-wall, 612 for the X-wall, and 1,130,634 for the cube, and that those solutions use 33, 275, and 13,842 different sets of cubies. This program works those numbers out from scratch.

The whole thing turns on one sentence of the exercise: the pictures use the skew projection $(x, y, z) \mapsto (30x - 42y,\ 14x + 10y + 45z)u$. That fixes which cubies you can see and which ones hide behind them, and everything else follows.

```
package main
import (
    "flag"
    "fmt"
    "sort"
    "strings"
    cells "github.com/sjnam/dancing-cells"
)
⟨ Declarations 5 ⟩
⟨ Functions 4 ⟩
func main() {
    ⟨ Read the command line 2 ⟩
    ⟨ Work out the unit cube's footprint 8 ⟩
    ⟨ Do what the mode asks 3 ⟩
}
```

2. Two modes. The census just counts the placements of the seven pieces in the $3 \times 3 \times 3$ cube, which is a good way to check the piece definitions against the book: §7.2.2.1 says there are 688 of them and 11,520 solutions. The other mode does the real work, one façade at a time.

⟨ Read the command line 2 ⟩ ≡

```
mode := flag.String("mode", "facade", "census or facade")
shape := flag.String("shape", "all", "wwall, xwall, cube, or all")
under := flag.Int("under", 0, "how many levels below the floor may be used")
verbose := flag.Bool("v", false, "list the visible and hidden cubies")
flag.Parse()
```

This code is used in section 1.

3. ⟨ Do what the mode asks 3 ⟩ ≡

```
switch *mode {
case "census":
    ⟨ Report the census of the cube 24 ⟩
case "facade":
    for _, s := range shapes() {
        if *shape ≡ "all" ∨ *shape ≡ s.name {
            ⟨ Work out one façade 32 ⟩
        }
    }
default:
    panic("unknown␣mode␣" + *mode)
}
```

This code is used in section 1.

4. The skew projection. A cubie is the unit cube whose least corner is at integer (x, y, z), and the picture puts it at $(30x - 42y,\ 14x + 10y + 45z)$. So the three axes go to $(30, 14)$, $(-42, 10)$ and $(0, 45)$: z straight up, x to the right and a little up, y to the left and a little up.

The line of sight is the kernel of that map. Solving $30x = 42y$ and $14x + 10y + 45z = 0$ gives $x = 7s$, $y = 5s$, $z = -148s/45$, so the kernel is spanned by $(315, 225, -148)$ and the viewer stands the other way, at $(-315, -225, 148)$. Depth is the component along that direction: the larger, the nearer.

Two things worth noticing. The viewer sees a cubie's top face and its $-x$ and $-y$ faces, which is exactly the three-faced look of the book's pictures. And the kernel is a lattice vector, so two cubies *can* project to the same place—but only 315 steps apart, far outside any board we care about. Within our region the projection is one-to-one on cubies, and that is what makes the whole exercise well posed: a picture determines its visible cubies exactly.

⟨ Functions 4 ⟩ ≡

```
func proj(p pt)(int, int) {
    return 30 * p.x - 42 * p.y, 14 * p.x + 10 * p.y + 45 * p.z
}
func depth(p pt) int {
    return -315 * p.x - 225 * p.y + 148 * p.z
}
```

This code is also defined in sections 9, 10, 12, 13, 15, 17, 19, 20, 21, 22, 23, 25, 26, 29, 31.

This code is used in section 1.

5. ⟨ Declarations 5 ⟩ ≡

```
type pt struct { x, y, z int }
```

This code is also defined in sections 6, 7, 18.

This code is used in section 1.

6. What the viewer sees. To decide which cubies show, I paint the picture and look. The outline of one cubie is the convex hull of its eight projected corners—a hexagon—and I sample that hexagon on a grid *sub* times finer than a projection unit. A cubie is visible if it is the nearest thing over at least one sample.

The projection is linear, so every cubie's hexagon is the same shape, just moved. I work out the samples of the hexagon at the origin once and shift them; since the projected corner of a cubie has integer coordinates, the shift is an exact whole number of samples and no rounding creeps in.

⟨ Declarations 5 ⟩ +≡

```
const sub = 4   // samples per projection unit
var unit []span   // the footprint of one cubie, relative to its own corner
```

7. ⟨ Declarations 5 ⟩ +≡

```
type span struct { row, lo, hi int }   // one scan line: columns lo..hi
```

8. The hull of the eight corners, then a scan line at a time. A convex polygon meets each horizontal line in one interval, so I only need the smallest and largest crossing.

⟨ Work out the unit cube's footprint 8 ⟩ ≡

```
var corner [][2]int
for dx := 0; dx < 2; dx++ {
    for dy := 0; dy < 2; dy++ {
        for dz := 0; dz < 2; dz++ {
            corner = append(corner, [2]int{})   // filled in just below
            x, y := proj(pt{dx, dy, dz})
            corner[len(corner) − 1] = [2]int{x, y}
        }
    }
}
unit = footprint(hull(corner))
```

This code is used in section 1.

9. ⟨ Functions 4 ⟩ +≡

```
func hull(ps [][2]int) [][2]int {
    sort.Slice(ps, func(i, j int) bool {
        if ps[i][0] ≠ ps[j][0] {
            return ps[i][0] < ps[j][0]
        }
        return ps[i][1] < ps[j][1]
    })
    cross := func(o, a, b [2]int) int {
        return (a[0] − o[0]) ∗ (b[1] − o[1]) − (a[1] − o[1]) ∗ (b[0] − o[0])
    }
    var h [][2]int
    for _, p := range ps {
        for len(h) ≥ 2 ∧ cross(h[len(h) − 2], h[len(h) − 1], p) ≤ 0 {
            h = h[:len(h) − 1]
        }
        h = append(h, p)
    }
    lower := len(h) + 1
    for i := len(ps) − 2; i ≥ 0; i−− {
        for len(h) ≥ lower ∧ cross(h[len(h) − 2], h[len(h) − 1], ps[i]) ≤ 0 {
            h = h[:len(h) − 1]
        }
        h = append(h, ps[i])
    }
    return h[:len(h) − 1]
}
```

10. The sample at row r and column c stands for the point $((c+\frac{1}{2})/sub,\ (r+\frac{1}{2})/sub)$, so I work in halves to keep everything in integers.

⟨ Functions 4 ⟩ +≡

```
func footprint(poly [][2]int) []span {
    lo, hi := poly[0][1], poly[0][1]
    for _, p := range poly {
        if p[1] < lo {
            lo = p[1]
        }
        if p[1] > hi {
            hi = p[1]
        }
    }
    var out []span
    for row := lo * sub - 1; row ≤ hi * sub + 1; row++ {
        y2 := 2 * row + 1   // twice the sample's height, times sub
        var l, r int
        got := false
        ⟨ Cross this scan line with every edge 11 ⟩
        if ¬got {
            continue
        }
        c0, c1 := ceilHalf(l), floorHalf(r)
        if c1 ≥ c0 {
            out = append(out, span{row, c0, c1})
        }
    }
    return out
}
```

11. An edge from a to b meets the line $y = y_2/(2\,sub)$ when y_2 lies between $2\,sub\,a_y$ and $2\,sub\,b_y$. I keep the crossing as twice its x coordinate times sub, again to stay in integers.

⟨ Cross this scan line with every edge 11 ⟩ ≡

```
for i := range poly {
    a, b := poly[i], poly[(i + 1) % len(poly)]
    ay, by := 2 * sub * a[1], 2 * sub * b[1]
    if ay == by {
        continue
    }
    if (ay <= y2 && y2 < by) || (by <= y2 && y2 < ay) {
        x := 2 * sub * a[0] + (y2 - ay) * 2 * sub * (b[0] - a[0])/(by - ay)
        if !got {
            l, r, got = x, x, true
        } else {
            if x < l {
                l = x
            }
            if x > r {
                r = x
            }
        }
    }
}
```

This code is used in section 10.

12. Turning a doubled coordinate back into the first and last sample columns it covers. A sample at column c sits at doubled coordinate $2c + 1$.

⟨ Functions 4 ⟩ +≡

```
func ceilHalf(x2 int) int {
    c := x2/2
    for 2 * c + 1 < x2 {
        c++
    }
    for 2 * (c - 1) + 1 >= x2 {
        c--
    }
    return c
}
func floorHalf(x2 int) int {
    c := x2/2
    for 2 * c + 1 > x2 {
        c--
    }
    for 2 * (c + 1) + 1 <= x2 {
        c++
    }
    return c
}
```

13. The picture itself: for every sample, the depth of the nearest cubie over it. That is all I need, because depth identifies a cubie uniquely here.

⟨ Functions 4 ⟩ +≡

```
func picture(s map[pt]bool) map[[2]int]int {
    buf := map[[2]int]int{}
    for p := range s {
        px, py := proj(p)
        d := depth(p)
        for _, sp := range unit {
            row := sp.row + py * sub
            for col := sp.lo + px * sub; col ≤ sp.hi + px * sub; col++ {
                k := [2]int{row, col}
                if old, ok := buf[k]; ¬ok ∨ d > old {
                    buf[k] = d
                }
            }
        }
    }
    return buf
}
```

14. A cubie shows if it wins some sample outright.

⟨ Decide which cubies are visible 14 ⟩ ≡

```
buf := picture(solid)
seen := map[pt]bool{}
for p := range solid {
    px, py := proj(p)
    d := depth(p)
    for _, sp := range unit {
        row := sp.row + py * sub
        for col := sp.lo + px * sub; col ≤ sp.hi + px * sub; col++ {
            if buf[[2]int{row, col}] ≡ d {
                seen[p] = true
                break
            }
        }
        if seen[p] {
            break
        }
    }
}
```

This code is used in section 32.

15. What may hide behind. Once the visible cubies are fixed, any other cubie may be added freely so long as it changes nothing—that is, so long as every sample above it already belongs to something nearer. Infinitely many cubies pass that test, since the space behind a wall goes on forever, but only a few can matter: the seven pieces have 27 cubies in all, so if v of them are spoken for, at most $27 - v$ are left to hide.

That is where the answer's word "distance" comes in, and the answer does not define it. I recovered it from the two lists the answer prints. Going one step in $+x$ or in $+y$ moves a cubie away from the viewer; a change of height counts as a step too. So the distance of a hidden cubie is

$$(q_x - v_x) + (q_y - v_y) + q\_z - v\_z,$$

minimized over the visible cubies v with $v_x \le q_x$ and $v_y \le q_y$. With that reading the W-wall's two shells come out as the answer's own $\{241, 242, 251, 252, 331, 332, 421, 422, 521, 522\}$ and $\{341, 342, 351, 352, 431, 432, 531, 532, 621, 622\}$, and the X-wall's profile comes out as the answer's own $(9, 7, 6, 3, 3, 2, 1)$. Two independent fits, so I am confident it is what he meant.

⟨ Functions 4 ⟩ +≡

```
func hidden(buf map[[2]int]int, q pt) bool {
    px, py := proj(q)
    d := depth(q)
    for _, sp := range unit {
        row := sp.row + py * sub
        for col := sp.lo + px * sub; col ≤ sp.hi + px * sub; col++ {
            if old, ok := buf[[2]int{row, col}]; ¬ok ∨ old < d {
                return false
            }
        }
    }
    return true
}
func distance(seen map[pt]bool, q pt) int {
    best := −1
    for v := range seen {
        if v.x > q.x ∨ v.y > q.y {
            continue
        }
        k := (q.x − v.x) + (q.y − v.y) + abs(q.z − v.z)
        if best < 0 ∨ k < best {
            best = k
        }
    }
    return best
}
func abs(a int) int {
    if a < 0 {
        return −a
    }
    return a
}
```

16. I sweep a box big enough to hold everything within reach and keep the cubies that hide. The floor is at $z = 1$ unless `-under` says otherwise; the answer mentions leaving out cases like 450 as "below ground," and notes separately that allowing underground cubies opens ten more ways to fake the cube.

⟨ Collect the cubies that may hide behind 16 ⟩ ≡

```
pool := []pt{}
for x := -3; x < 18; x++ {
    for y := -3; y < 18; y++ {
        for z := 1 - *under; z < 10; z++ {
            q := pt{x, y, z}
            if seen[q] ∨ ¬hidden(buf, q) {
                continue
            }
            if d := distance(seen, q); d ≥ 0 ∧ d ≤ 27 - len(seen) {
                pool = append(pool, q)
            }
        }
    }
}
sort.Slice(pool, func(i, j int) bool { return less(pool[i], pool[j]) })
```

This code is used in section 32.

17. ⟨ Functions 4 ⟩ +≡

```
func less(a, b pt) bool {
    if a.x ≠ b.x {
        return a.x < b.x
    }
    if a.y ≠ b.y {
        return a.y < b.y
    }
    return a.z < b.z
}
```

18. The seven Soma pieces. Piet Hein's seven shapes, in the order and with the names of (39): bent, ell, tee, skew, L-twist, R-twist, claw. The first is the only tricube; the rest have four cubies each, so the set has $3 + 6 \cdot 4 = 27$ cubies, exactly a $3 \times 3 \times 3$ cube.

I take the coordinates of the bent piece and of the ell straight from the text, which says that `1␣000␣001␣010` is one of the placements of the bent piece and that the ell's six canonical placements are shifts of $(000, 010, 020, 100)$. The other five I wrote down from the picture. The check that they are right comes at the end of this section.

⟨ Declarations 5 ⟩ +≡

```
var soma = [7]struct {
    name string
    cubie []pt
}{
    {"bent", []pt{{0,0,0},{0,0,1},{0,1,0}}},
    {"ell", []pt{{0,0,0},{0,1,0},{0,2,0},{1,0,0}}},
    {"tee", []pt{{0,0,0},{0,1,0},{0,2,0},{1,1,0}}},
    {"skew", []pt{{0,0,0},{0,1,0},{1,1,0},{1,2,0}}},
    {"L-twist", []pt{{0,0,0},{1,0,0},{1,1,0},{1,1,1}}},
    {"R-twist", []pt{{0,0,0},{1,0,0},{1,1,0},{0,0,1}}},
    {"claw", []pt{{0,0,0},{1,0,0},{0,1,0},{0,0,1}}},
}
```

19. Twenty-four rotations, generated by turning about two axes until nothing new appears. Reflections are *not* included: the twists are chiral and a real Soma set cannot mirror them. (I tried allowing reflections once, to see what would happen, and the cube promptly reported 54,048 solutions instead of 11,520—a useful reminder that the check below is worth having.)

⟨ Functions 4 ⟩ +≡

```
func turns() []func(pt) pt {
    rx := func(p pt) pt { return pt{p.x, -p.z, p.y} }
    ry := func(p pt) pt { return pt{p.z, p.y, -p.x} }
    seen := map[[3]pt]bool{}
    out := []func(pt) pt{}
    todo := []func(pt) pt { func(p pt) pt { return p } }
    for len(todo) > 0 {
        var next []func(pt) pt
        for _, f := range todo {
            key := [3]pt{f(pt{1,0,0}), f(pt{0,1,0}), f(pt{0,0,1})}
            if seen[key] {
                continue
            }
            seen[key] = true
            out = append(out, f)
            g := f
            next = append(next, func(p pt) pt { return rx(g(p)) })
            next = append(next, func(p pt) pt { return ry(g(p)) })
        }
        todo = next
    }
    return out
}
```

20. Sliding a shape to the origin, so that two descriptions of the same solid can be compared. The cubies come back sorted.

⟨ Functions 4 ⟩ +≡

```
func slide(cs []pt) []pt {
    mx, my, mz := cs[0].x, cs[0].y, cs[0].z
    for _, c := range cs {
        if c.x < mx {
            mx = c.x
        }
        if c.y < my {
            my = c.y
        }
        if c.z < mz {
            mz = c.z
        }
    }
    out := make([]pt, len(cs))
    for i, c := range cs {
        out[i] = pt{c.x − mx, c.y − my, c.z − mz}
    }
    sort.Slice(out, func(i, j int) bool { return less(out[i], out[j]) })
    return out
}
```

21. And writing the slid shape down as a string, which is what I use as a key.

⟨ Functions 4 ⟩ +≡

```
func normal(cs []pt) string {
    var b strings.Builder
    for _, c := range slide(cs) {
        fmt.Fprintf(&b, "%d.%d.%d␣", c.x, c.y, c.z)
    }
    return b.String()
}
```

22. The distinct orientations of one piece: turn it every way and keep the ones not seen before. The bent piece has 12, the ell 24, the claw only 8.

⟨ Functions 4 ⟩ +≡

```
func shapesOf(k int) [][]pt {
    seen := map[string]bool{}
    var out [][]pt
    for _, f := range turns() {
        cs := make([]pt, len(soma[k].cubie))
        for i, c := range soma[k].cubie {
            cs[i] = f(c)
        }
        key := normal(cs)
        if seen[key] {
            continue
        }
        seen[key] = true
        out = append(out, slide(cs))
    }
    sort.Slice(out, func(i, j int) bool { return normal(out[i]) < normal(out[j]) })
    return out
}
```

23. Every way of dropping a piece into a set of allowed cells.

```
⟨ Functions 4 ⟩ +≡
  func places(k int, region map[pt]bool) [][]pt {
    var base []pt
    for c := range region {
      base = append(base, c)
    }
    sort.Slice(base, func(i, j int) bool { return less(base[i], base[j]) })
    seen := map[string]bool{}
    var out [][]pt
    for _, sh := range shapesOf(k) {
      for _, b := range base {
        put := make([]pt, len(sh))
        ok := true
        for i, c := range sh {
          put[i] = pt{b.x + c.x, b.y + c.y, b.z + c.z}
          if ¬region[put[i]] {
            ok = false
            break
          }
        }
        if ¬ok {
          continue
        }
        sort.Slice(put, func(i, j int) bool { return less(put[i], put[j]) })
        key := normal(put) + fmt.Sprint(put[0])
        if ¬seen[key] {
          seen[key] = true
          out = append(out, put)
        }
      }
    }
    return out
  }
```

24. Here is the check that the seven shapes are the right seven. Section 7.2.2.1 says the cube gives 688 options and 11,520 solutions, and both numbers come out.

⟨ Report the census of the cube 24 ⟩ ≡

```
cube := map[pt]bool{}
for x := 0; x < 3; x++ {
    for y := 0; y < 3; y++ {
        for z := 0; z < 3; z++ {
            cube[pt{x, y, z}] = true
        }
    }
}
total := 0
for k := range soma {
    n := len(places(k, cube))
    total += n
    fmt.Printf("␣␣%d␣%-8s␣%d␣cubies,␣%2d␣orientations,␣%3d␣placements\n",
        k + 1, soma[k].name, len(soma[k].cubie), len(shapesOf(k)), n)
}
fmt.Printf("options␣for␣the␣3x3x3␣cube:␣%d␣(the␣book␣says␣688)\n", total)
in, _ := problem(cube, Λ)
fmt.Printf("solutions:␣%d␣(the␣book␣says␣11520)\n", count(in))
```

This code is used in section 3.

25. ⟨ Functions 4 ⟩ +≡

```
func count(in string) int {
    xc := cells.NewXCC()
    res := xc.Dance(strings.NewReader(in))
    n := 0
    for range res.Solutions {
        n++
    }
    return n
}
```

26. The exact cover problem. The seven pieces are primary items of multiplicity one, and so is every visible cubie: the façade must show all of them. The cubies that may hide are secondary, since they may be used or not. Every placement of every piece inside the region becomes an option.

```
⟨ Functions 4 ⟩ +≡
  func problem(region map[pt]bool, optional []pt)(string, int) {
      opt := map[pt]bool{}
      for _, c := range optional {
          opt[c] = true
      }
      var req []pt
      for c := range region {
          if ¬opt[c] {
              req = append(req, c)
          }
      }
      sort.Slice(req, func(i, j int) bool { return less(req[i], req[j]) })
      var b strings.Builder
      ⟨ Write the item line 27 ⟩
      n := 0
      for k := range soma {
          for _, put := range places(k, region) {
              fmt.Fprintf(&b, "%d", k + 1)
              for _, c := range put {
                  fmt.Fprintf(&b, "␣%s", name(c))
              }
              b.WriteString("\n")
              n++
          }
      }
      return b.String(), n
  }
  func name(c pt) string { return fmt.Sprintf("c%d.%d.%d", c.x, c.y, c.z) }
```

27. ⟨ Write the item line 27 ⟩ ≡

```
  for k := range soma {
      fmt.Fprintf(&b, "%d␣", k + 1)
  }
  for _, c := range req {
      fmt.Fprintf(&b, "%s␣", name(c))
  }
  if len(optional) > 0 {
      b.WriteString("|")
      for _, c := range optional {
          fmt.Fprintf(&b, "␣%s", name(c))
      }
  }
  b.WriteString("\n")
```

This code is used in section 26.

28. Connected, and standing up. Solving the cover problem is not the end of it. The answer says to throw away the solutions "that are disconnected or violate the gravity constraint of exercise 333," so two tests remain.

Connectedness is the easy one: the 27 cubies must hang together face to face.

Gravity took longer to pin down, because exercise 333 gives pictures rather than a rule, and its structures include a cantilever and a mushroom—so overhangs are clearly allowed. What is not allowed is a piece floating in mid-air. Since the pieces are rigid, the test belongs to the piece and not to the cubie: a piece stands if one of its cubies is on the floor, or if one of them rests directly on a cubie of another piece. Equivalently, a piece stands unless you could slide it straight down without hitting anything.

That reading is confirmed by the cube, where it gives 1,130,634—the answer's number to all seven digits.

⟨ Decide whether the cubies hang together 28 ⟩ ≡

```
whole := true
{
    var start pt
    for p := range used {
        start = p
        break
    }
    reach := map[pt]bool{start: true}
    stack := []pt{start}
    for len(stack) > 0 {
        p := stack[len(stack) − 1]
        stack = stack[:len(stack) − 1]
        for _, q := range neighbours(p) {
            if used[q] ∧ ¬reach[q] {
                reach[q] = true
                stack = append(stack, q)
            }
        }
    }
    whole = len(reach) ≡ len(used)
}
```

This code is used in section 35.

29. ⟨ Functions 4 ⟩ +≡

```
func neighbours(p pt) [6]pt {
    return [6]pt{{p.x + 1, p.y, p.z}, {p.x − 1, p.y, p.z},
        {p.x, p.y + 1, p.z}, {p.x, p.y − 1, p.z},
        {p.x, p.y, p.z + 1}, {p.x, p.y, p.z − 1}}
}
```

30. ⟨ Decide whether every piece is held up 30 ⟩ ≡

```
stands := true
{
    owner := map[pt]int{ }
    for i, ps := range chunk {
        for _, p := range ps {
            owner[p] = i
        }
    }
    for i, ps := range chunk {
        held := false
        for _, p := range ps {
            if p.z ≡ floor {
                held = true
                break
            }
            if j, ok := owner[pt{p.x, p.y, p.z − 1}]; ok ∧ j ≠ i {
                held = true
                break
            }
        }
        if ¬held {
            stands = false
            break
        }
    }
}
```

This code is used in section 35.

31. The three façades. The W-wall's coordinates are the ones answer 326 uses, a zigzag of nine columns three cubies high. The X-wall is two walls of five crossing at the middle; I settled on that shape by trying every pair of crossing walls that makes 27 cubies and keeping the one whose visible count and hidden-cubie profile agree with the answer. The cube is the cube.

```
⟨ Functions 4 ⟩ +≡
    func shapes( ) []struct {
        name string
        foot [][2]int
    }{
        return []struct {
            name string
            foot [][2]int
        }{
            {"wwall", [][2]int{{5,1},{4,1},{3,1},{3,2},{3,3},
                {2,3},{1,3},{1,4},{1,5}}},
            {"xwall", [][2]int{{3,1},{3,2},{3,3},{3,4},{3,5},
                {1,3},{2,3},{4,3},{5,3}}},
            {"cube", [][2]int{{1,1},{1,2},{1,3},{2,1},{2,2},{2,3},
                {3,1},{3,2},{3,3}}},
        }
    }
```

32. One façade from beginning to end.

```
⟨ Work out one façade 32 ⟩ ≡
    solid := map[pt]bool{ }
    for _, f := range s.foot {
        for z := 1; z ≤ 3; z++ {
            solid[pt{f[0], f[1], z}] = true
        }
    }
    ⟨ Decide which cubies are visible 14 ⟩
    ⟨ Collect the cubies that may hide behind 16 ⟩
    ⟨ Report what the geometry gives 33 ⟩
    ⟨ Solve, filter, and report 35 ⟩
```

This code is used in section 3.

33.

```
⟨ Report what the geometry gives 33 ⟩ ≡
    fmt.Printf("\n%s: %d cubies, %d visible, %d may hide\n",
        s.name, len(solid), len(seen), len(pool))
    if *verbose {
        ⟨ List the cubies 34 ⟩
    }
```

This code is used in section 32.

34. ⟨ List the cubies 34 ⟩ ≡

```
var hid []pt
for p := range solid {
    if ¬seen[p] {
        hid = append(hid, p)
    }
}
sort.Slice(hid, func(i, j int) bool { return less(hid[i], hid[j]) })
fmt.Printf("␣␣invisible␣cubies␣of␣the␣solid:")
for _, p := range hid {
    fmt.Printf("␣%d%d%d", p.x, p.y, p.z)
}
fmt.Println()
shell := map[int][]pt{}
for _, q := range pool {
    d := distance(seen, q)
    shell[d] = append(shell[d], q)
}
for d := 1; d ≤ 27; d++ {
    if len(shell[d]) ≡ 0 {
        continue
    }
    fmt.Printf("␣␣distance␣%d␣(%2d):", d, len(shell[d]))
    for _, q := range shell[d] {
        fmt.Printf("␣%d%d%d", q.x, q.y, q.z)
    }
    fmt.Println()
}
```

This code is used in section 33.

35. ⟨ Solve, filter, and report 35 ⟩ ≡

```
region := map[pt]bool{}
for p := range seen {
    region[p] = true
}
for _, q := range pool {
    region[q] = true
}
in, nopt := problem(region, pool)
floor := 1 - *under
var all, conn, both int
sets := map[string]bool{}
xc := cells.NewXCC()
res := xc.Dance(strings.NewReader(in))
for sol := range res.Solutions {
    all++
    ⟨ Read the solution back as cubies 36 ⟩
    ⟨ Decide whether the cubies hang together 28 ⟩
    ⟨ Decide whether every piece is held up 30 ⟩
    ⟨ Tally this solution 37 ⟩
}
fmt.Printf("␣␣%d␣options,␣%d␣solutions,␣%d␣connected,\n", nopt, all, conn)
fmt.Printf("␣␣%d␣of␣those␣also␣standing,␣filling␣%d␣sets␣of␣cubies\n",
    both, len(sets))
```

This code is used in section 32.

36. ⟨ Read the solution back as cubies 36 ⟩ ≡

```
used := map[pt]bool{}
var chunk [][]pt
for _, o := range sol {
    var ps []pt
    for _, it := range o[1:] {
        var p pt
        if _, err := fmt.Sscanf(it, "c%d.%d.%d", &p.x, &p.y, &p.z); err == Λ {
            used[p] = true
            ps = append(ps, p)
        }
    }
    chunk = append(chunk, ps)
}
```

This code is used in section 35.

37. ⟨ Tally this solution 37 ⟩ ≡

```
if whole {
    conn++
    if stands {
        both++
        var ks []string
        for p := range used {
            ks = append(ks, name(p))
        }
        sort.Strings(ks)
        sets[strings.Join(ks, "␣")] = true
    }
}
```

This code is used in section 35.

38. Index.

Hollywood Soma